DBMS

Topics

Unit 4:

- An overview of NoSQL
- Characteristics of NoSQL
- NoSQL storage types
- Advantages and Drawbacks of NoSQL
- Case Study: Application definition Requirement Analysis
- Implementation using MongoDB
- Database Queries
- Writing Queries

NOSQL!

NoSQL databases are currently a hot topic in some parts of computing, with over a hundred different NoSQL databases.

Document Database	Graph Databases
Couchbase, MarkLogic, mongoDB	Neo4j, InfiniteGraph (The Distributed Graph Database)
Wide Column Stores	**Key-Value Databases**
redis, amazon DynamoDB, AEROSPIKE, riak	accumulo, HYPERTABLE, Cassandra, APACHE HBASE, Amazon SimpleDB

RDBMS CHARACTERISTICS

- Data stored in columns and tables
- Relationships represented by data
- Data Manipulation Language
- Data Definition Language
- Transactions
- Abstraction from physical layer
- Applications specify what, not how
- Physical layer can change without modifying applications
  - Create indexes to support queries
  - In Memory databases

NO SQL?

- NoSQL stands for:
  - No Relational
  - No RDBMS
  - Not Only SQL
- NoSQL is an umbrella term for all databases and data stores that don’t follow the RDBMS principles
  - A class of products
  - A collection of several (related) concepts about data storage and manipulation
  - Often related to large data sets

NOSQL VS. SQL

SQL DB	NoSQL DB
Examples: DB2, MySQL, Oracle, Postgress, SQL server	Examples: CouchDb MongoDB, RavenDb, Redis, Cassandra, Hbase, Neo4j,BigTable
These are called RDBMS.	These are called not only SQL database.
Based on ACID properties i.e. Atomicity, Consistency, Isolation and Durability	Based on CAP properties i.e. (Consistency, Availability and Partition tolerance)
These are table based database i.e. the data are stored in a table with rows and columns.	These databases are document based, key-value pairs or graph based etc.
These are standard schema based (predefined schema)	These are not standard schema based(dynamic schema)
These are scaled vertically. Load can be managed by increasing CPU, RAM etc in the same server.	These are scaled horizontally. A few servers can be added to manage large traffic.
Not preferred for large/big data sets.	Preferred for large/big data sets.
Preferred for complex query execution	Not preferred for complex query execution

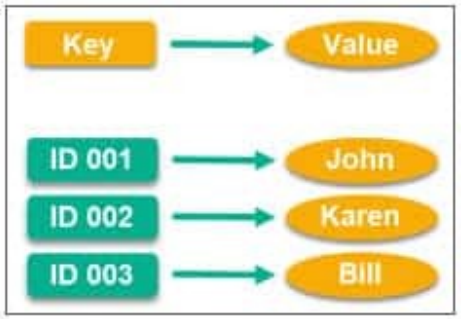
SQL Databases

Table

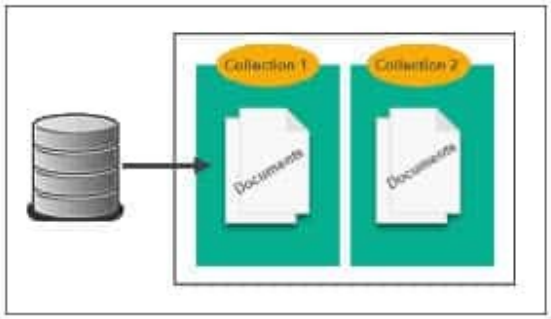
ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

NoSQL Databases

Key-value

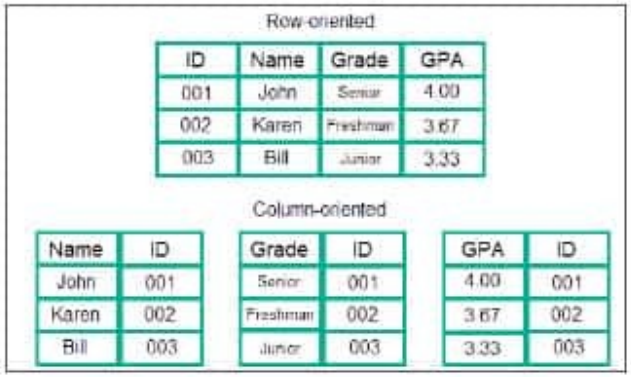


Document



Graph

Wide-column



Row-oriented

ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented

Name	ID
John	001
Karen	002
Bill	003

Grade	ID
Senior	001
Freshman	002
Junior	003

GPA	ID
4.00	001
3.67	002
3.33	003

WHERE DOES NOSQL COME FROM?

- Non-relational DBMSs are not new
- But NoSQL represents a new incarnation
  - Due to massively scalable Internet applications
  - Based on distributed and parallel computing
- Development
  - Starts with Google
  - First research paper published in 2003
  - Continues also thanks to Lucene's developers/Apache (Hadoop) and Amazon (Dynamo)
  - Then a lot of products and interests came from Facebook, Netfix, Yahoo, eBay, Hulu, IBM, and many more

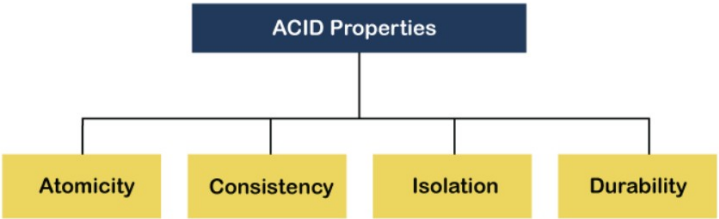
Defining NoSQL

- In computing, NoSQL (mostly interpreted as "**not only SQL**") is a broad class of database management systems identified by its non-adherence to the widely used relational database management system model;
- that is, NoSQL databases are not primarily built on tables, and as a result, generally do not use SQL for data manipulation

What NoSQL is and What NoSQL is not

- NoSQL is a generic term used to refer to any data store that does not follow the traditional RDBMS model
- Specifically, the data is non-relational and it does not use SQL as the query language.
- It is used to refer to the databases that attempt to solve the problems of scalability and availability against that of atomicity or consistency.

TRANSACTIONS – ACID PROPERTIES



- Atomic – All of the work in a transaction completes (commit) or none of it completes
- Consistent – A transaction transforms the database from one consistent state to another consistent state. Consistency is defined in terms of constraints.
- Isolated – The results of any changes made during a transaction are not visible until the transaction has committed.
- Durable – The results of a committed transaction survive failures

Difficulties with ACID property

- **quite incompatible** with availability and performance on applications of web-scale.
- For example:

Difficulties with ACID property

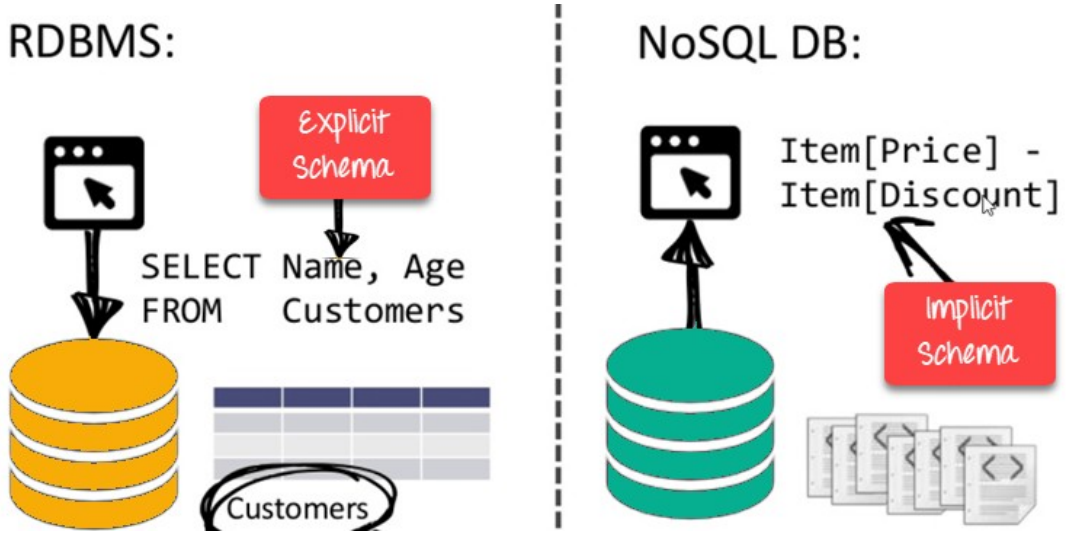
- Amazon will work **very slow** in the following scenario.
- For example:
- If I proceed to buy a book and a transaction is on, it will l**ock a part of the database**, specifically the inventory, and **every other person in the world will have to wait** until I complete my transaction.

Solution: BASE

Look beyond ACID to something called **BASE** , coined by Eric Brewer

- **Basic availability: Each request is guaranteed a response—successful or failed execution.**
- **Soft state: The state of the system may change over time, at times without any input (for eventual consistency).**
- **Eventual consistency: The database may be momentarily inconsistent but will be consistent eventually**

Why NoSQL(1)?



Schemaless data representation:

- Almost all NoSQL implementations offer schemaless data representation.
- This means that you don't have to think too far ahead to define a structure and you can continue to evolve over time—including adding new fields or even nesting the data, for example, in case of JSON representation.

Why NoSQL? (2)

Development time: **Reduced development time** because one doesn't have to deal with complex SQL queries.

Speed: Even with the small amount of data that you have, if you can deliver in milliseconds rather than hundreds of milliseconds — especially over mobile and other intermittently connected devices— you have much higher probability of winning users over.

Plan ahead for scalability: your application can be quite elastic—it can handle sudden spikes of load. Of course, you win users over straightaway

List of NoSQL Databases

Document	**Key-Value**	**XML**	**Column**	**Graph**
MongoDB	Redis	BaseX	BigTable	Neo4J
CouchDB	Membase	eXist	Hadoop / HBase	FlockDB
RavenDB	Voldemort		Cassandra	InfiniteGraph
Terrastore	MemcacheDB		SimpleDB	
			Cloudera	

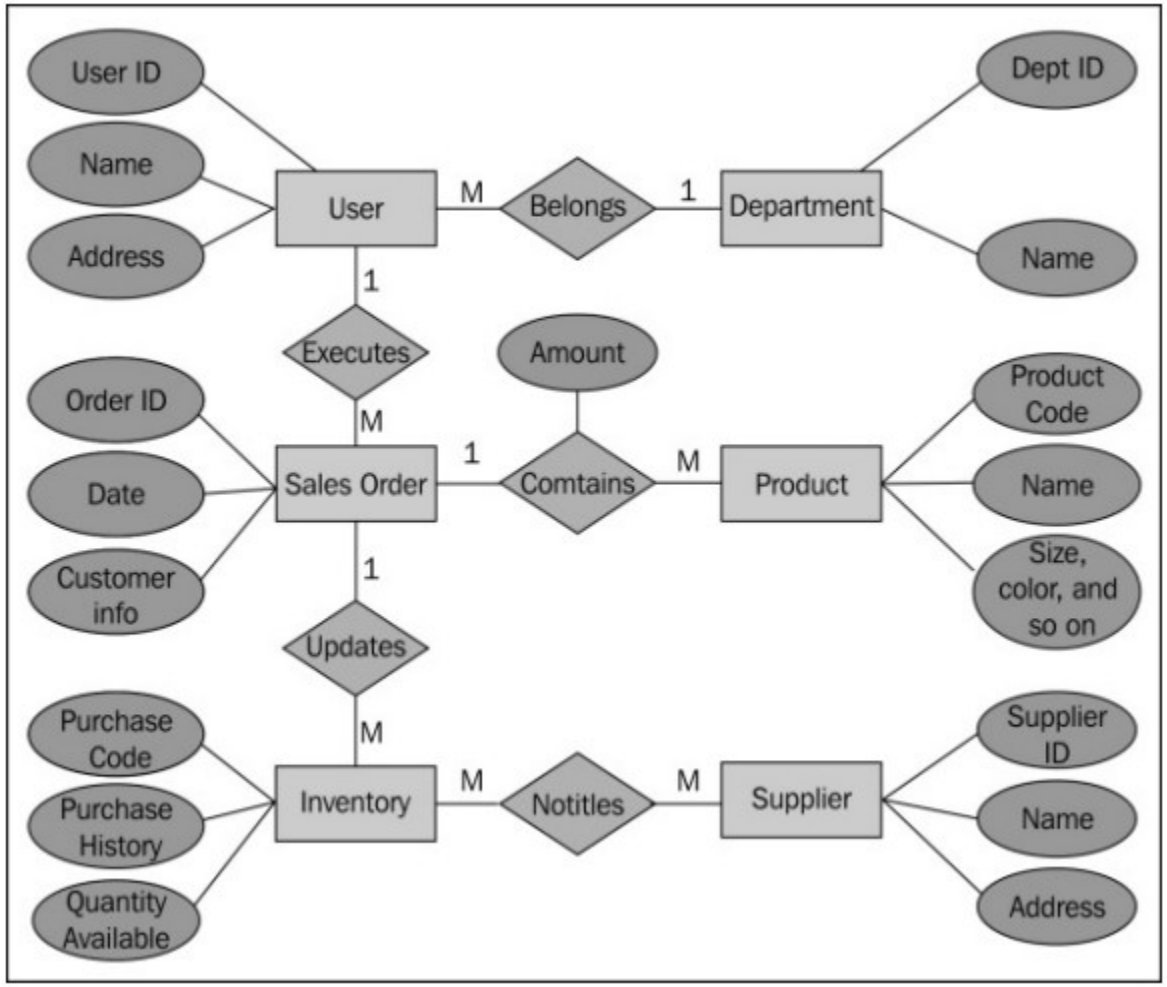
Details regarding the types f databases will be illustrated later….

Characteristics of NoSQL

Let’s have a look back at the designing phase of RDBMS approach (steps):

- Identify actors
- Define models
- Define entities
- Define relationships
- Program database and application
- Iterate

Example


User ID
Name
Address
User
M
Belongs
1
Department
Dept ID
Name
1
Executes
Amount
Order ID
M
Product Code
Date
Sales Order
1
Comtains
M
Product
Name
Customer info
1
Size, color, and so on
Updates
Purchase Code
M
Supplier ID
Purchase History
Inventory
M
Notitles
M
Supplier
Name
Quantity Available
Address

Example(Cont.)

- Few of the **actors** identified include buyer, employee, purchaser, administrator, office address, shipping address, supplier address, item in inventory, and supplier.
- They may be mapped to a **model** UserProfile and there may be subclasses as required—**Administrator and PointOfSalesUser.** Some of the other models include Department, Role, Product, Supplier, Address, PurchaseOrder, and Invoice.
- Simplistically, a database table may map each actor to a model.
- **Foreign keys will be used to define the object relationships—one-to-many between Department and UserProfile, many-to-many** between Role and UserProfile, and PurchaseOrder and Product.

Challenges with RBMS approach

- **Difficult to maintain database—schema,** programmability, business continuity process a.k.a. availability, and other aspects
- How to **expand** their web presence and enable online orders??
- The company acquires another company and now needs **to integrate the two database systems.**
- The company grows big and has to handle **hundreds of millions of queries a day across** the country
- The company plans to provide API integration for aggregators to retrieve and process their data

Demands to satisfy Functional Requirements!!!

- **Schema flexibility:This will be needed during future enhancements and integration with external applications**
- **Complex queries : so-called JOIN queries which are not only difficult to implement and maintain but also take substantial database resources to execute**
- **Data update: Updating data across tables is probably one of the more complex scenarios especially if they are to be a part of the transaction.**
- **Scalability**

NoSQL approach

Following are the approaches followed by NoSQL to find the solutions to the challenges faced by database systems

1. **Schema flexibility**
2. **No Complex queries**
3. **Data update**
4. **Scalability**

NoSQL approach(1)

1. **Schema flexibility:** **Columnoriented DBMS** store data as columns as opposed to rows in RDBMS. This allows flexibility of adding one or more columns as required, on the fly. Similarly, **document stores that allow storing semi-structured** data are also good options.

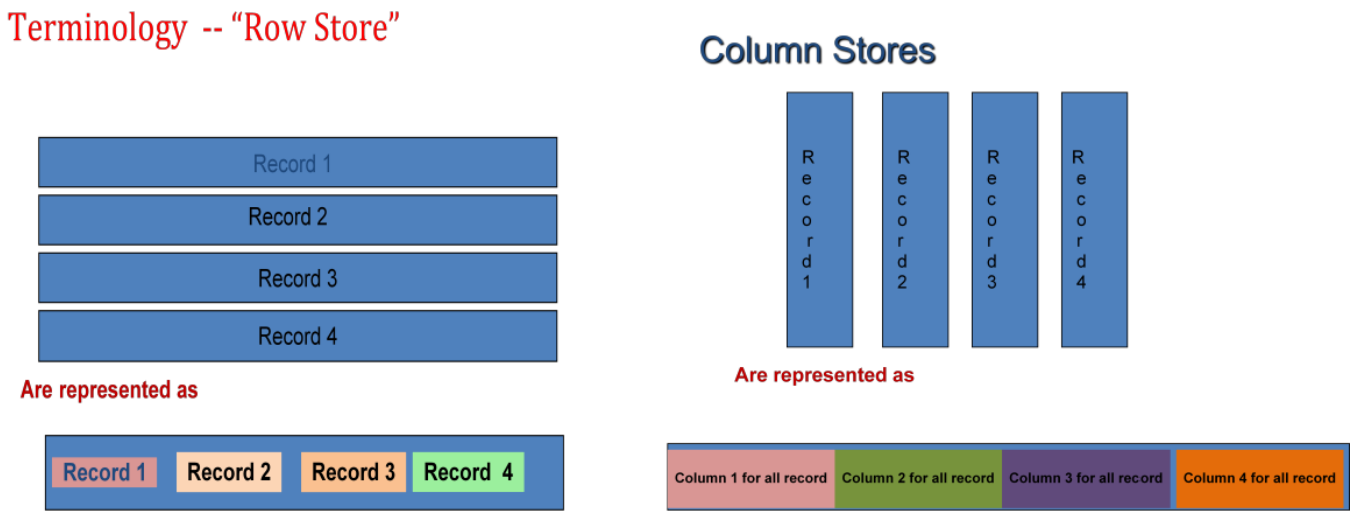


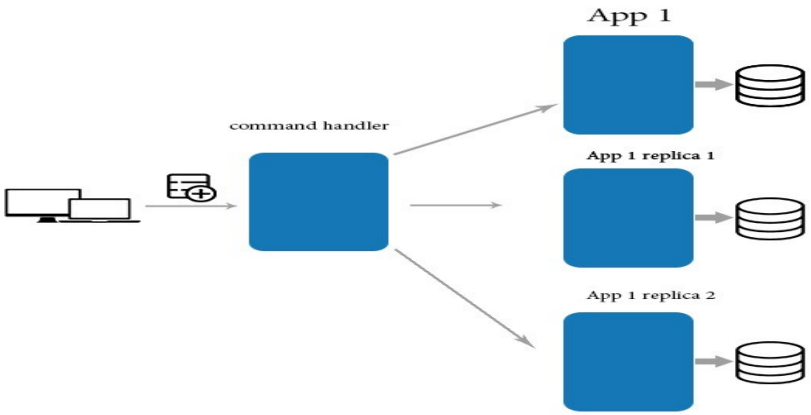
Figure 1: Row store and Column store architectures

NoSQL approach(2)

2. No Complex queries: NoSQL databases do not have support for relationships or foreign keys. There are no complex queries. There are no JOIN statements.

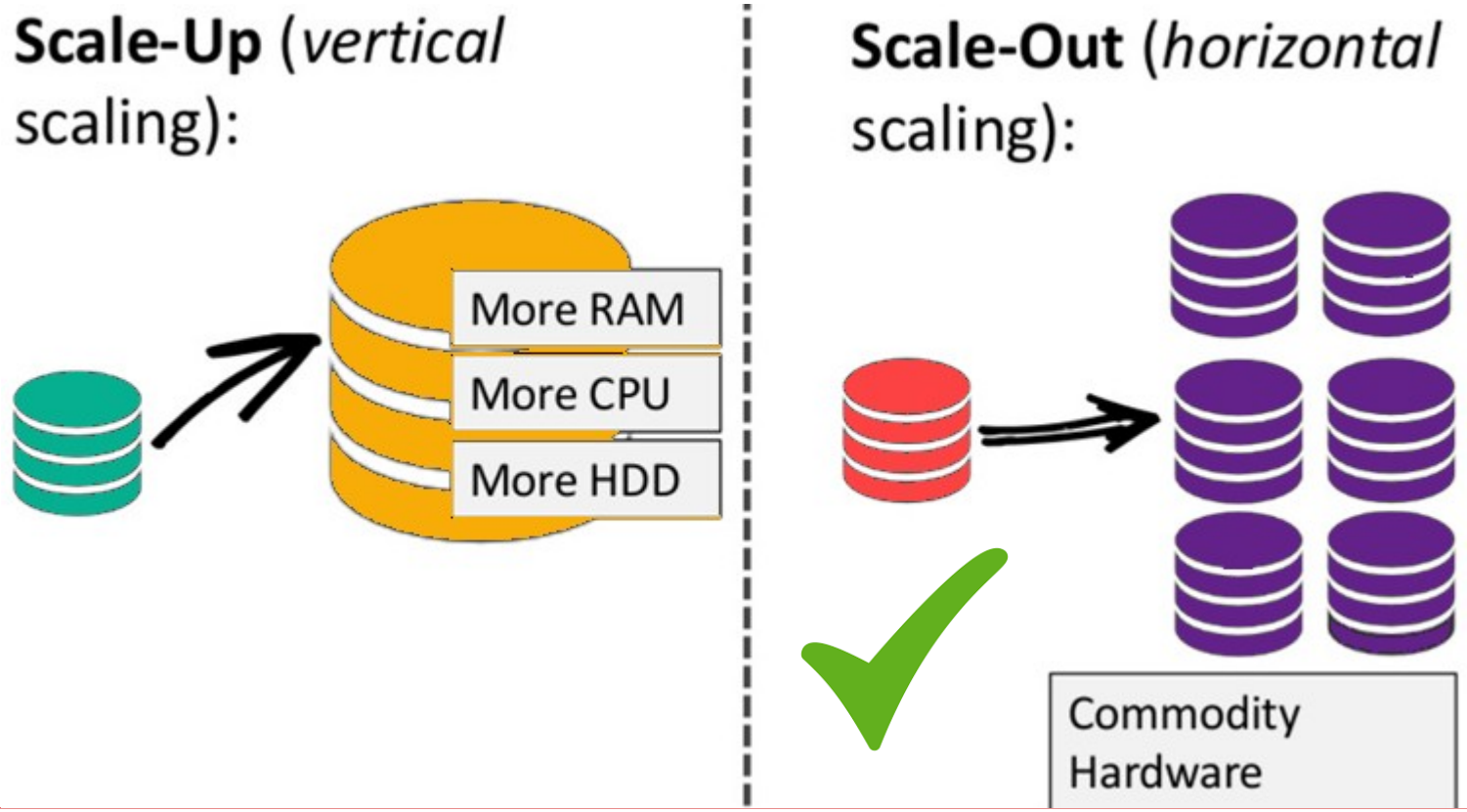
3. Data update: Data update and synchronization across nodes within a datacenter has a different set of requirements as compared to synchronizing across multiple datacenters.

- Need minimum latency
- NoSQL solutions offer great synchronization options.



NoSQL approach(3)

4. Scalability:. A lot of complexity that is required for transaction oriented RDBMS does not exist in ACID non-compliant NoSQL databases.



NOSQL Storage TYPES

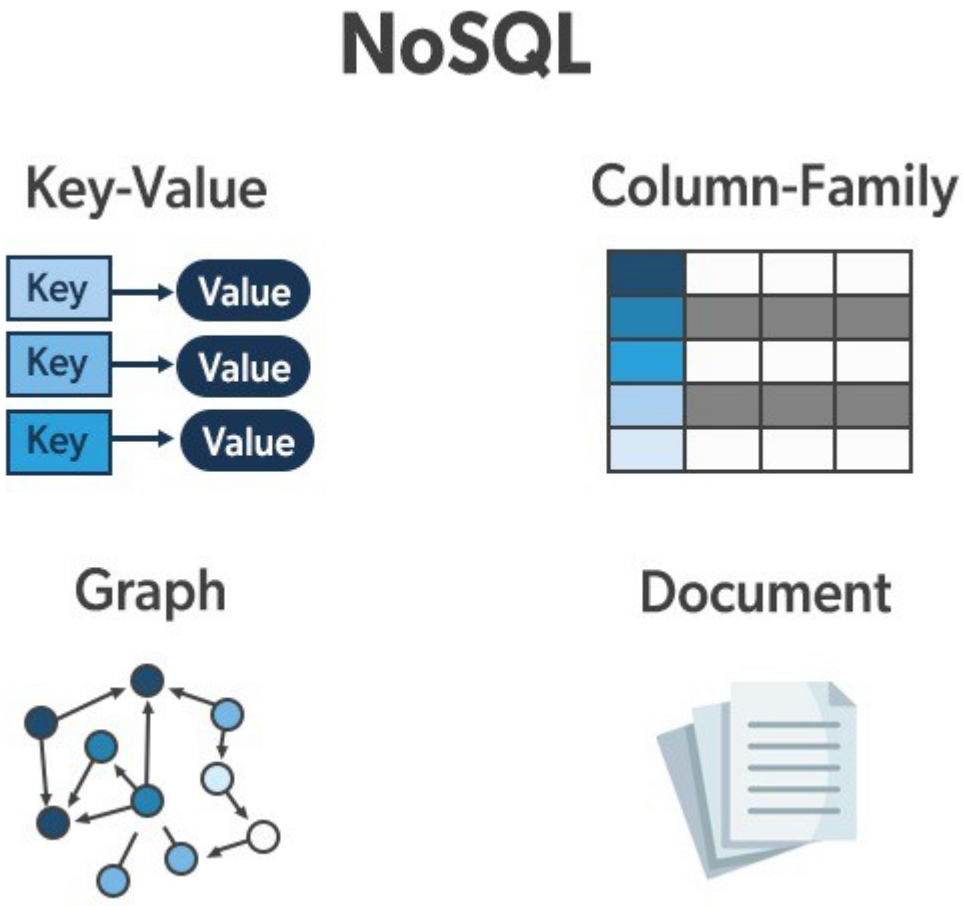
NOSQL Storage TYPES

- The NoSQL databases are **categorized on the basis of how the data is stored**.

- Because of the need to provide curated information from large volumes, generally in near real-time, NoSQL mostly **follows a horizontal structure**.

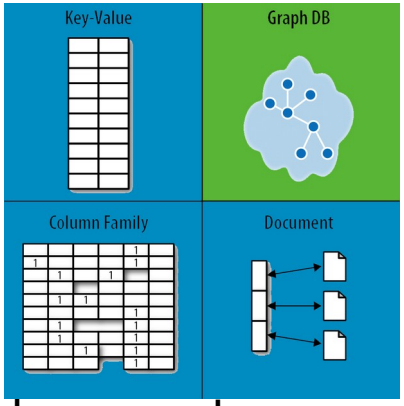
- They are **optimized for insert and retrieve operations** on a large scale with built-in capabilities for replication and clustering.

NOSQL Storage TYPES



NOSQL Storage TYPES

Discussing NoSQL databases is complicated because there are a variety of types:



- **Column-oriented databases**
  - Optimized for queries over large datasets, and store columns of data together, instead of rows
- **Document databases:**
  - pair each key with a complex data structure known as a document.
- **Key-Value Store :**
  - are the simplest NoSQL databases. Every single item in the database is stored
    as an attribute name (or 'key'), together with its value.
- **Graph Databases :**
  - are used to store information about networks of data, such as social connections.

Column-oriented databases

EmployeeID	FirstName	LastName	Age	Salary
SM1	Anuj	Sharma	45	10000000
MM2	Anand		34	5000000
T3	Vikas	Gupta	39	7500000
E4	Dinesh	Verma	32	2000000

Row Oriented Storage

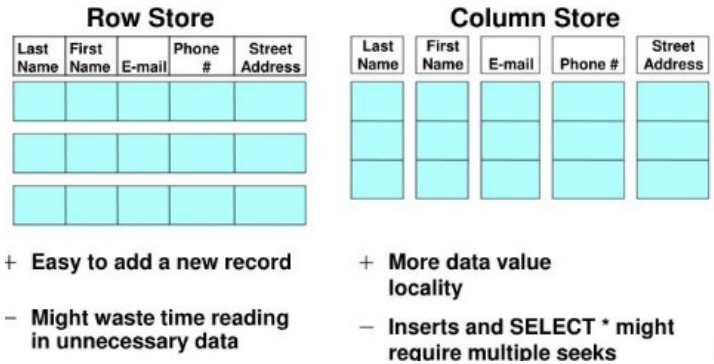
```
SM1,Anuj,Sharma,45,10000000
MM2,Anand,,34,5000000
T3,Vikas,Gupta,39,7500000
E4,Dinesh,Verma,32,2000000
```

Column Oriented Storage

```
SM1,MM2,T3,E4
Anuj,Anand,Vikas,Dinesh
Sharma,,Gupta,Verma,
45,34,39,32
10000000,5000000,7500000,2000000
```

COLUMN-ORIENTED STORES

- Data are stored in a column-oriented way
  - Data efficiently stored
  - Avoids consuming space for storing nulls
  - Columns are grouped in column-families
  - Data isn't stored as a single table but is stored by column families
  - Unit of data is a set of key/value pairs
    - Identified by "row-key"
    - Ordered and sorted based on row-key
- Notable for:
  - Google's Bigtable (used in all Google's services)
  - HBase (Facebook, StumbleUpon, Hulu, Yahoo!, ...)



Advantages of Column Oriented Database

- Allow **adding columns over time** without having to worry about filling in default values for the existing rows for the new columns.
- This gives f**lexibility in model and entity design** allowing one to account for new columns in future for unforeseen scenarios and new requirements.
- Similarly, when new values are applied for either all rows at once or with same-column filters, these **databases will allow partial data access without touching unrelated columns** and be much faster in execution.
- Since columns will be of uniform type and mostly (except in cases of variable-length strings) of the same length, there are possibilities of **efficient storage in terms of size**.

DOCUMENT DATABASES, JSON

```
{
    _id: ObjectId("51156a1e056d6f966f268f81"),
    type: "Article",
    author: "Derick Rethans",
    title: "Introduction to Document Databases with MongoDB",
    date: ISODate("2013-04-24T16:26:31.911Z"),
    body: "This arti…"
},
{
    _id: ObjectId("51156a1e056d6f966f268f82"),
    type: "Book",
    author: "Derick Rethans",
    title: "php|architect's Guide to Date and Time Programming
with PHP",
    isbn: "978-0-9738621-5-7"
}
```

Different Details

DOCUMENT DATABASES

- **Documents**
  - Loosely structured sets of key/value pairs in documents, e.g., XML, JSON
  - Encapsulate and encode data in some standard formats or encodings
  - Are addressed in the database via a unique key
  - Documents are treated as a whole, avoiding splitting a document into its constituent name/value pairs
- Allow documents retrieving by keys or contents
- Notable for:
  - MongoDB (used in FourSquare, Github, and more)
  - CouchDB (used in Apple, BBC, Canonical, Cern, and more)

DOCUMENT DATABASES

- The central concept is the notion of a "**document" which corresponds to a row in RDBMS.**
- A document comes in some standard formats like **JSON (BSON)**.
- Documents are addressed in the database via a **unique *key*** that represents that document.
- The database offers an API or query language that retrieves documents based on their contents.
- Documents are **schema free**, i.e., different documents can have structures and schema that differ from one another. (An RDBMS requires that each row contain the same columns.)

DOCUMENT DATABASES - Advantages

- **content is schemaless, or at best loosely defined. This is very useful in web-based applications where there is a need for storing different types of content that may evolve over time.**
  - For example, for a grocery store, information about the users, inventory and orders can be stored as simple JSON or XML documents.
- it may or may not be possible to **retrieve or update a record partially.** If it is possible to do so, there is a great advantage.
- **Searching across multiple entity types is far more trivial compared to doing so in traditional RDBMS or even in column-oriented databases. Because, now, there is no concept of tables —which is essentially nothing more than a schema definition—one can query across the records, irrespective of the underlying content or schema or in other words, the query is directly against the entire database**
- **JSON-based stores are easy to define what can be called as projections . Each top-level key for the JSON object may be the entity's projection across other parts of the system thereby allowing the schema to evolve over time with backward compatibility.**

Example: MongoDB

- create a collection

```
> db.createCollection("userprofile");
```

- Inserting a record into userprofile collection

```
> db.userprofile.insert({
    "me": {
      "id": "document-uuid",
      ...
    },
    "personal": {
      "firstName": "Alice",
      ...
    }
});
```

Example: MongoDB

-Update

To update a record, use the update method that expects **two arguments.**

- The first argument is a **query to filter the record to be updated**.
- The second argument provides **details about the updated values.**

```
> db.userprofile.update( {
    "me.id": "uuid-to-search-for"
  }, {
    "$set": {
      "personal.lastName": "Taylor"
    }
  }
);
```

Where clause of SQL

Set of Update Query

find() method

To retrieve records, we can use any of the attributes:

```
> db.userprofile.find( {
    "personal.firstName": "Bob"
  }
);
```

remove() method

To **delete** a record or records, all that is required is to be able to query for appropriate attribute or attributes.

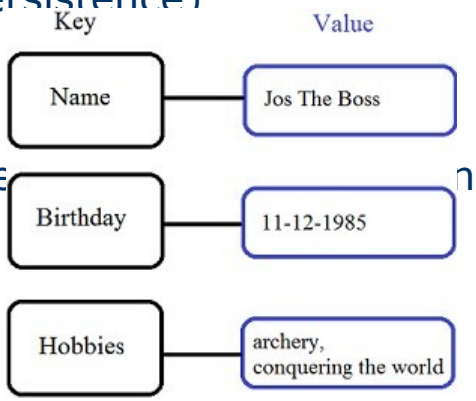
For example, to remove profiles whose age is greater than 30 and the city of thier personal address is Madrid:

```
> db.userprofile.remove({
    "personal.age": { $gt: 30 },
    "personal.address.city": "Madrid"
  }
);
```

KEY/VALUE STORES

- Store data in a **schema-less way**
- -storage of a **value against a key.**
- Similar to a document store,
- there is no need for a schema to be enforced on the value
- Notable for:
  - Redis (in-memory, with dump or command-log persistence)
  - Memcached (in-memory)
  - MemcacheDB (built on Memcached)

    Berkley DB Voldemort (open source impleme namo)



KEY/VALUE STORES(2)

- **Difference with Document DB**

- Unlike a document store that can create a key when a new document is inserted, a **key-value store requires the key to be specified**
- Unlike a document store where the value can be indexed and queried, for a key-value store, the **value is opaque and as such, the key must be known to retrieve the value**

KEY/VALUE STORES-Advantage

-**Key-value stores** are optimized for querying against keys.

- As such, they serve great in memory caches

Memcached and Redis support expiry for the keys—sliding or absolute— after which the entry is evicted from the store.

Examples(1)

Set or update value against a key:

```
SET company "My Company"          //String
HSET alice firstName "Alice"      //Hash – set field value
HSET alice lastName "Matthews"    //Hash – set field value
LPUSH "alice:sales" "10" "20"     //List create/append
LSET "alice:sales" "0" "4"        //List update
SADD "alice:friends" "f1" "f2"    //Set – create/update
SADD "bob:friends" "f2" "f1"      //Set – create/update
```

Examples(2)

- Set operations:

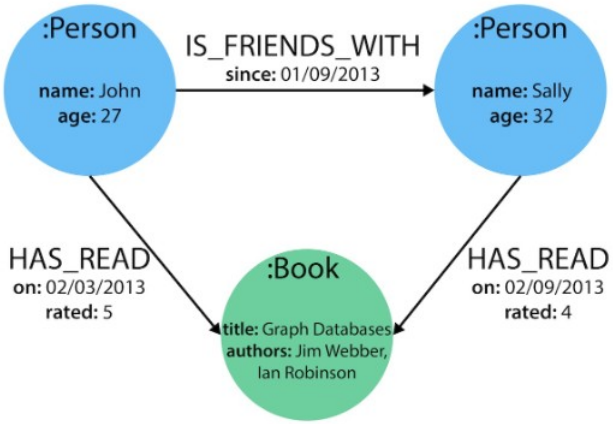
```
//Intersection – Get mutual friends of Alice and Bob
SINTER "alice:friends" "bob:friends"

//Difference – Friends in Alice's list absent in Bob's
SDIFF "alice:friends" "bob:friends"

//Union – All friends that need invitation in their marriage
SUNION "alice:friends" "bob:friends"
```

- List operations:

```
//Pop the first item, or return null
POP "key:name"

//Blocking pop – pop the first item, or wait until timeout or next
is available (check across lists – l1, l2, l3)

BLPOP l1 l2 l3

//Pop item from list1, append to list2 and return the value

RPOPLPUSH list1 list2
```

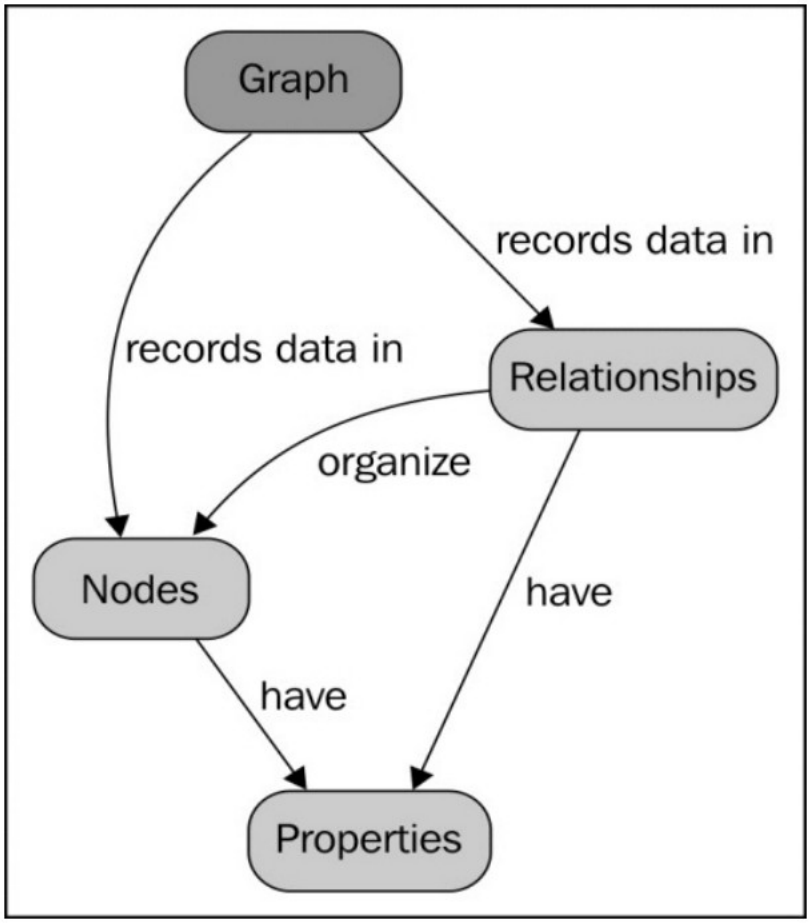
GRAPH DATABASES

- Graph-oriented
- Everything is stored as an edge, a node or an attribute.
- Each node and edge can have any number of attributes.
- Both the nodes and edges can be labelled.
- Labels can be used to narrow searches.



GRAPH DATABASES

- Relationships are represented as graphs.
- There can be multiple links between two nodes in a graph—representing the multiple relationships that the two nodes share
- Example: social relationships between people, transport links between places



Advantages

- Graph databases can be considered as special purpose NoSQL databases **optimized for relation-heavy data.**

- **Easy representation, retrieval and manipulation of relationships between the entities in the system.**

- It is not uncommon to store data in a document store and relationships in a graph database.

EXAMPLES

```
//Assume that we get the underlying database service somehow
GraphDatabaseService db = ...

Node node1 = db.createNode();
node1.setProperty("documentId", "alice");

Node node2 = db.createNode();
node2.setProperty("documentId", "bob");

RelationType friendRel = new RelationType() {
   public String name() { return "friend"; }
};

Relationship reln = node1.createRelationshipTo(node2,
                                                 friendRel);
reln.setProperty("initatedBy", "alice");
reln.setProperty("createdOn", "1980-01-01T07:30+0530");
```

Comparing the models

Feature	Column Oriented	Document Store	Key Value Store	Graph
Table-like schema support (columns)	Yes	No	No	Yes
Complete update/fetch	Yes	Yes	Yes	Yes
Partial update/fetch	Yes	Yes	Yes	No
Query/Filter on value	Yes	Yes	No	Yes
Aggregates across rows	Yes	No	No	No
Relationships between entities	No	No	No	Yes
Cross-entity view support	No	Yes	No	No
Batch fetch	Yes	Yes	Yes	Yes
Batch update	Yes	Yes	Yes	No

Advantages and Drawbacks

Advantages and Drawbacks

We will take **three different representative application requirements** and discuss advantages and disadvantages of using NoSQL

1. Transactional application
2. Computational application
3. Web-scale application

1.Transactional application

- This type of application has data that is highly **relational in nature.**
- An **example** of this is the application at the point-of-sales at a **grocery shop** or an enterprise resource management application. The key here is data consistency.

Entity schema requirements

- For a transactional application, general requirements for the entity schema include:
- Highly structured definition—properties, types, and constraints, if applicable
- Ability to define relationships—parent versus child keys
- Schema does not evolve or vary too much over time.

Data access requirements

- From a data access perspective, we would have the following requirements:
- Consistency—any read should return latest updated data
- More often than not, the entire record (row) will be retrieved
- Cross-entity (table) data access may be frequent

1.Transactional application (Contd…)

What NoSQL can do

For a transactional application, NoSQL can help as follows:

Column-oriented database helps define structure. If need be, it can be changed over time.

Document-oriented database can help implement JOIN or create views.

What NoSQL cannot do

Using NoSQL will result in a few limitations for a transactional application: Inability to define relationships Absence of transactions

Unavailability of ACID properties in operations

Using Cassandra or CouchDB can be overkill if we compare them to, for example, MySQL or PostgreSQL—they do not unleash their true power in a single-machine installation

Absence of support for JOIN and cross-entity query, though document-oriented stores support it by way of MapReduce but the efforts may be substantial as the queries get complex

Decision

Choose RDBMS rather than NoSQL. The disadvantages outweigh the advantages

2. Computational application

This type of application does a lot of computation in the application

An example of this type of application is a **survey application** or a **sales record management** application. The key here is that the computation is across the records but on a subset of the properties or columns available

Entity schema requirements

For a computational application, general requirements on the entity schema include:

Structured schema definition—properties, types, and constraints, if applicable.

Schema does not evolve or vary too much over time.

Data access requirements: From a data access perspective, we would have the following requirements:

- Partial record access.
- Vertical, that is, column-wise processing.
  - For example, for an entity holding the data of daily revenues and expenses at a given location of operation, one would compute across the revenue column and/or expense column more often than working on the entire row.
- Cross-entity data access is infrequent.
  - For example, to create the final balance sheet for a location, I may use the location ID once to get the details about the location.
- Consistency—data should be consistent. However, in some cases, minor latencies are allowed.
  - For example, since the reports may be generated daily rather than in real-time, the user is happy working with day-old data

2. Computational application (Contd…)

What NoSQL can do : For a computational application, NoSQL can help as follows:

Column-oriented databases would help define rigorous schema.

Document or keyvalue databases can still be used.

For example, JSON formats can be used to define a formal schema. Just that these stores cannot enforce the schema.

They (column-oriented, key-value as well as document stores) can provide speed and scale while fetching partial data.

Document stores coupled with MapReduce processing can help performing computation right up close to the data-tier thereby substantially increasing the speed of execution. You do not want unnecessary data to be floating across the network.

Document stores can help implement JOIN or create views.

What NoSQL cannot do

Using NoSQL will result in a few limitations for a computational application:

Defining relationships can be tricky. Except for graph databases, these relationships must be maintained in application.

Because relationships do not exist in the database, data can be inconsistent —even if momentarily.

2. Computational application (Contd…)

Decision

To conclude, it is a tough choice between RDBMS and NoSQL databases in this case. If the application is, say, an enterprise application **wherein the number of records will be limited to around**, for example, a **hundred million or lower**, **RDBMS can just serve it right**, though NoSQL can also be used. For any size less than a million records, NoSQL can have overheads in terms of setup and tuning; while for **over a few hundred million records**, **RDBMS will taper down in performance**.

3. Web-Scale application

This last application type is probably more relevant today in consumer applications, whether they are completely web-based or mobile-native apps or a combination of both.

An example of this application is a **web analytics application** or **a social microblogging platform**. The key here is the scale of operation and staleness of data.

Entity schema requirements

For a web-scale application, general requirements on the entity schema include:

Structured schema definition

The ability to change schema over time without affecting existing records in any manner—in extreme case, latent schema

Relationships may be optional at database layer and can be pushed to application layer mainly because of the low density

Data access requirements: From a data access perspective, we would have the following requirements:

- Partial record access
- Speed of operation execution—CRUD
- Inconsistent data—for a short moment—is tolerable

3. Web-Scale application (Contd…)

What NoSQL can do

For a web-scale application, NoSQL can help as follows:

Document stores would be a fantastic choice for latent schema.

It can provide scale of operations because it does not implement ACID operations

but mostly provide BASE properties

What NoSQL cannot do

Note that we do not have ACID constraints—one of the main reasons why NoSQL was invented.

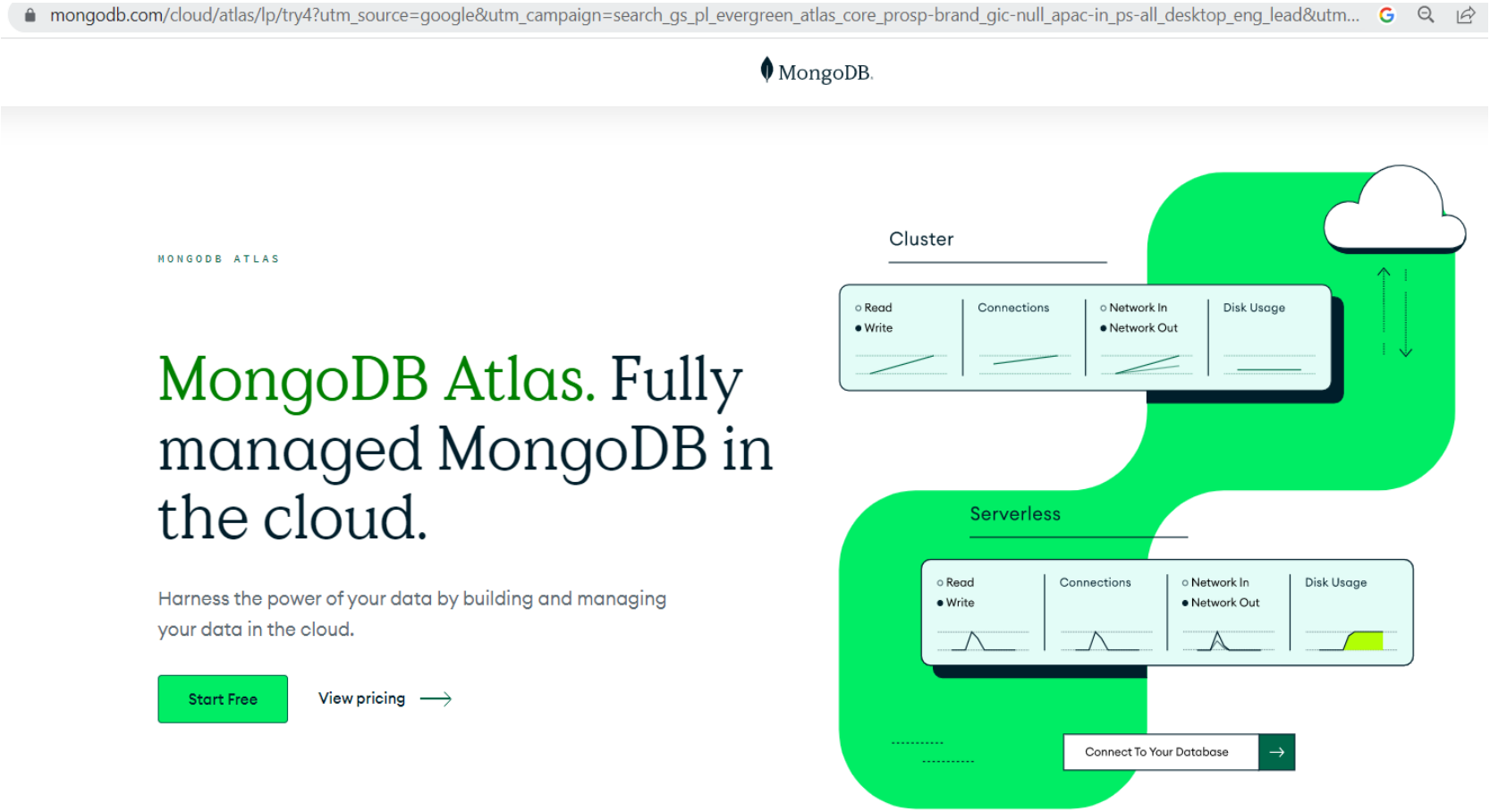
Decision

Use NoSQL

Implementation using MongoDB

MONGODB –DOCUMENT DB

MongoDB Atlas



Why MongoDB?

- Open Source
- Distributed
- Fast In-Place Updates
- Replication
- Full Index Support
- Rich Query Language
- Easy Scalability
- Auto sharding

JSON (Java Script Object Notation)

Sample JSON Document

```
{
FirstName: John,
LastName: Mathews,
ContactNo: [+123 4567 8900, +123 4444 5555]
}
```

Unique Identifier

Each JSON document should have a unique identifier. It is the _id key.

0	1	2	3	4	5	6	7	8	9	10	11
Timestamp				Machine ID			Process ID		Counter		

Terms Used in RDBMS and MongoDB

RDBMS	MongoDB
Database	Database
Table	Collection
Record	Document
Columns	Fields / Key Value pairs
Index	Index
Joins	Embedded documents
Primary Key	Primary key (_id is a identifier)

CREATE DATABASE IN MONGODB

use myDB;

Confirm the existence of your database

db;

```
Atlas atlas-mdgaz1-shard-0 [primary] DBMS_DEMO> use myDB;
switched to db myDB
Atlas atlas-mdgaz1-shard-0 [primary] myDB>
```

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db;
myDB
```

CRUD (CREATE, READ, UPDATE, DELETE) OPERATIONS

To create a collection by the name “Student”.

db.createCollection(“Student”);

=> sql equivalent
CREATE TABLE STUDENT(…);

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.createCollection("Student");
{ ok: 1 }
```

CREATE DATABASE IN MONGODB

use myDB;

```
Atlas atlas-mdgaz1-shard-0 [primary] DBMS_DEMO> use myDB;
switched to db myDB
Atlas atlas-mdgaz1-shard-0 [primary] myDB>
```

Drop Collection

To drop a collection by the name “Student”.

```
db.Student.drop();
```

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.drop();
true
```

Insert Data

```
db.Student.insert(
{_id:1,
StudName:"MichelleJacintha",
Grade:"VII",
Hobbies:"InternetSurfing"});
```

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.insert({_id:1,StudName:"MichelleJacintha",Grade:"VII",Hobbies:"InternetSurfing"});
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{ acknowledged: true, insertedIds: { '0': 1 } }
```

Bulk Insert

var mystudent =
[{_id:4, StudName:"saurav", Grade:"V", Hobbies:"Dance"},
{_id:5, StudName:"kumar", Grade:"VI", Hobbies:"Singing"}]

db.Student.insert(mystudent)

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> var mystudent = [ {_id:4, StudName:"saurav", Grade:"V", Hobbies:"Dance"},{_id:5, StudName:"kumar", Grade:"VI", Hobbies:"Singin

Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.insert(mystudent)
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{ acknowledged: true, insertedIds: { '0': 4, '1': 5 } }
```

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find()
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  },
  { _id: 4, StudName: 'saurav', Grade: 'V', Hobbies: 'Dance' },
  { _id: 5, StudName: 'kumar', Grade: 'VI', Hobbies: 'Singing' }
]
```

Update Method

ıdb.student.update({Studname:’Aryan Dravid’},{$set: {“Studname”:”Saurav”}})

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Students.update({_id:3,  StudName:"Aryan  David",  Grade:  "VII"},{$set:{Hobbies: "Skating"}},{upsert:true});
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: 3,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

Update or Insert

Insert the document for “Aryan David” into the Students collection only if it does not already exist in the collection. However, if it is already present in the collection, then update the document with new values. (Update his Hobbies from “Skating” to “Chess”.) Use “Update else insert” (if there is an existing document, it will attempt to update it, if there is no existing document then it will insert it).

db.Student.update({_id:3, StudName:"Aryan David", Grade: "VII"},{$set: {Hobbies: "Skating"}},{upsert:true});

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Students.update({_id:3,  StudName:"Aryan  David",  Grade:  "VII"},{$set:{Hobbies: "Skating"}},{upsert:true});
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: 3,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

View Data

View the entire table

db.Student.find()

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find()
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Same as

Select * from table;

Find Method (Search Based on Criteria)(1)

To search for documents from the “Students” collection based on certain search criteria.

db.Student.find({StudName:"Aryan David"});

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find({StudName:"Aryan  David"});
[
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Find Method (Search Based on Criteria)(2)

To display only the StudName and Grade from all the documents of the Students collection.

The identifier _id should be suppressed and NOT displayed.

db.Students.find({},{StudName:1,Grade:1,_id:0});

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find({},{StudName:1,Grade:1,_id:0});
[
  { StudName: 'MichelleJacintha', Grade: 'VII' },
  { Grade: 'VII', StudName: 'Aryan  David' }
]
```

Find Method (Search Based on Criteria)(3)

To find those documents where the Grade is set to ‘VII’

db.Students.find({Grade:{$eq:'VII'}})

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find({Grade:{$eq:'VII'}});
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Find Method (Search Based on Criteria)(4)

To find those documents from the Students collection where the Hobbies is set to either ‘Chess’ or is set to ‘Skating’.

db.Students.find ({Hobbies :{ $in: ['Chess','Skating']}})

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find ({Hobbies :{ $in: ['Chess','Skating']}})
[
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Find Method (Search Based on Criteria)(5)

To find documents from the Students collection where the StudName begins with “M”.

db.Students.find({StudName:/^M/});

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find({StudName:/^M/});
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  }
]
```

Find Method (Search Based on Criteria)(6)

To find documents from the Students collection where the StudName has an "e" in any position.

db.Students.find({StudName:/e/});

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find({StudName:/e/});
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  }
]
```

Count Method

To find the number of documents in the Students collection.

db.Student.count();

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.count();
DeprecationWarning: Collection.count() is deprecated. Use countDocuments or estimatedDocumentCount.
2
```

Sort Method

To sort the documents from the Students collection in the **ascending** order of StudName.

db.Students.find().sort({StudName:1})

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find().sort({StudName:1})
[
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  },
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  }
]
```

To sort the documents from the Students collection in the **descending** order of StudName.

db.Students.find().sort({StudName:-1})

```
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Remove data

```
db.Student.remove({"StudName":"saurav"})

db.student.find({"StudName":"saurav"});
```

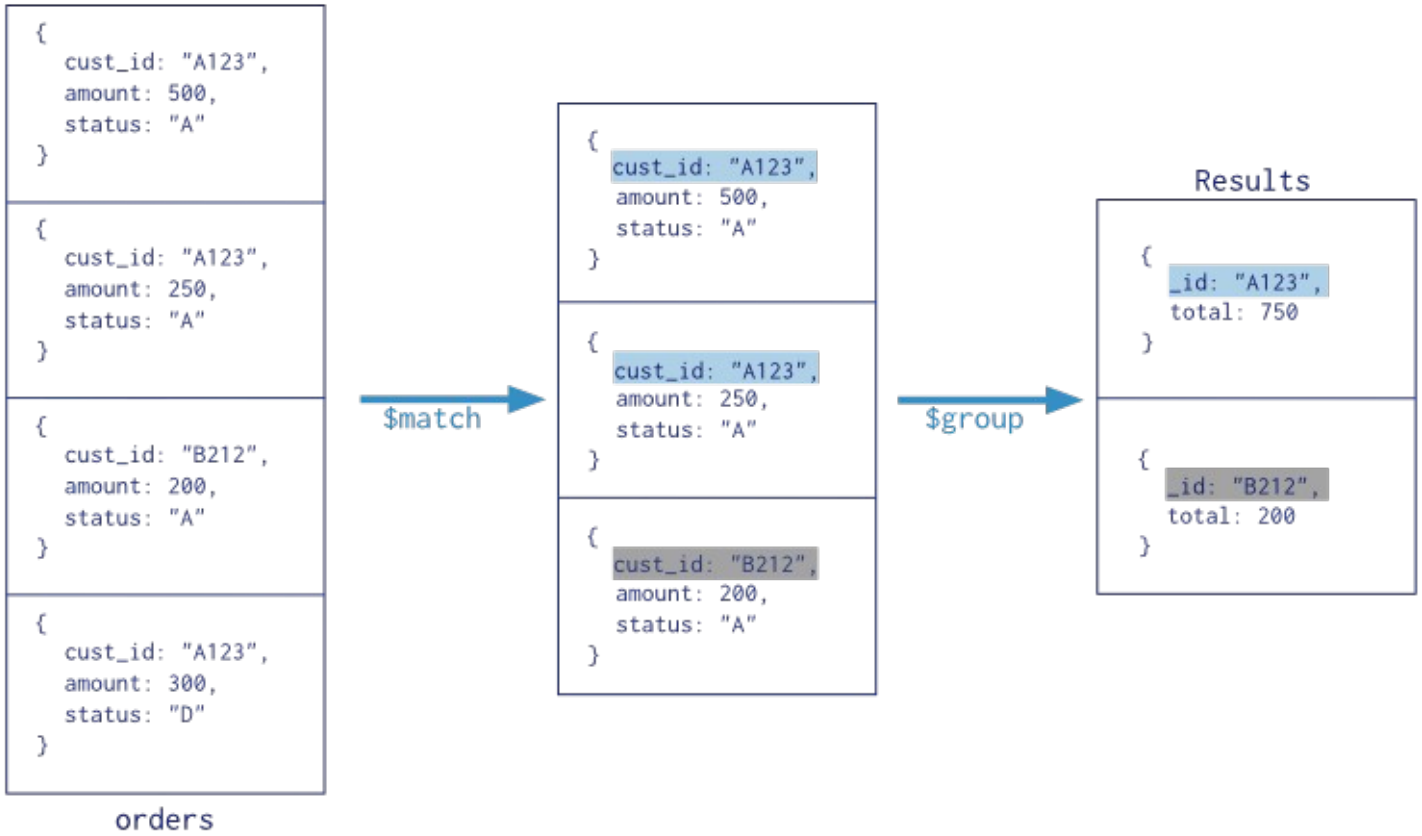
Display using Limit

db.Student.find().limit(2)

```
Atlas atlas-mdgaz1-shard-0 [primary] myDB> db.Student.find().limit(2)
[
  {
    _id: 1,
    StudName: 'MichelleJacintha',
    Grade: 'VII',
    Hobbies: 'InternetSurfing'
  },
  {
    _id: 3,
    Grade: 'VII',
    StudName: 'Aryan  David',
    Hobbies: 'Skating'
  }
]
```

Aggregate Function in MongoDb (example)

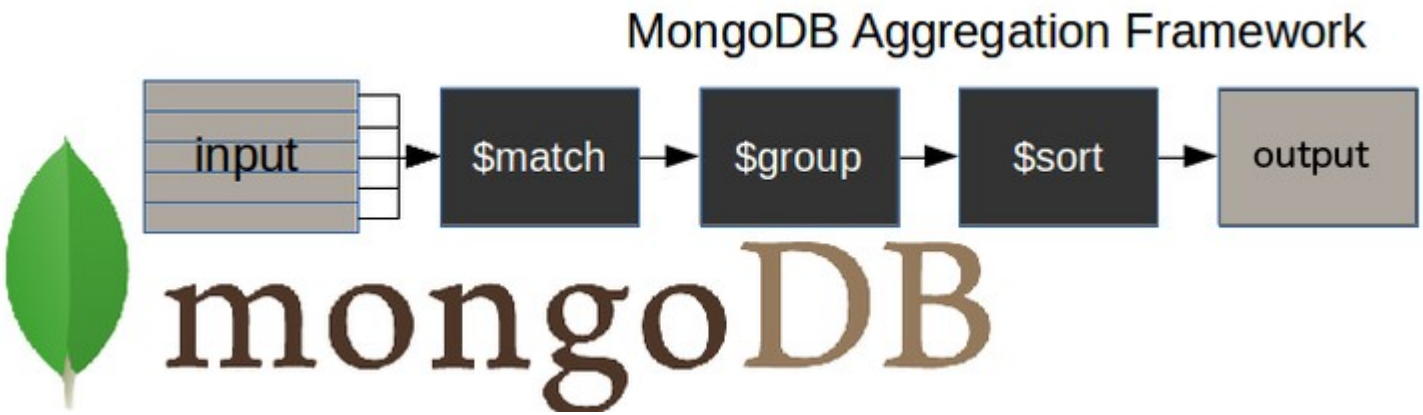
```
Collection
db.orders.aggregate(
    $match phase ──► { $match: { status: "A" } },
    $group phase ──► { $group: { _id: "$cust_id",total: { $sum: "$amount" } } }
)
```



Aggregation

- **Aggregation is a way of processing a large number of documents in a collection by means of passing them through different stages.**
- The stages make up what is known as a pipeline. The stages in a pipeline can filter, sort, group, reshape and modify documents that pass through the pipeline.
- One of the most common use cases of Aggregation is to calculate aggregate values for groups of documents.

How does the MongoDB aggregation pipeline work?



- **`$match` stage** – filters those documents we need to work with, those that fit our needs
- **`$group` stage** – does the aggregation job
- **`$sort` stage** – sorts the resulting documents the way we require (ascending or descending)

MongoDB aggregate pipeline syntax

`db.`*`collectionName`*`.aggregate(`*`pipeline, options`*`),`

- where *collectionName* – is the name of a collection,
- *pipeline – is an array that contains the aggregation stages,*
- *options – optional parameters for the aggregation*

Example

Assume the following collection

```
{
   _id: ObjectId(7df78ad8902c)
   title: 'MongoDB Overview',
   description: 'MongoDB is no sql database',
   by_user: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
   tags: ['mongodb', 'database', 'NoSQL'],
   likes: 100
},
{
   _id: ObjectId(7df78ad8902d)
   title: 'NoSQL Overview',
   description: 'No sql database is very fast',
   by_user: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
   tags: ['mongodb', 'database', 'NoSQL'],
   likes: 10
},
{
   _id: ObjectId(7df78ad8902e)
   title: 'Neo4j Overview',
   description: 'Neo4j is no sql database',
   by_user: 'Neo4j',
   url: 'http://www.neo4j.com',
   tags: ['neo4j', 'database', 'NoSQL'],
   likes: 750
},
```

1. Display a list stating how many tutorials are written by each user

```
> db.mycol.aggregate([{$group : {_id : "$by_user", num_tutorial : {$sum : 1}}}])
```

Output:

```
{ "_id" : "tutorials point", "num_tutorial" : 2 }
{ "_id" : "Neo4j", "num_tutorial" : 1 }
```

```
db.mycol.aggregate([
{$group : {_id : "$by_user",
num_tutorial : {$sum : "$likes"}}}])
```

```
db.mycol.aggregate([{$group : {_id :
"$by_user", num_tutorial : {$avg :
"$likes"}}}])
```

```
db.mycol.aggregate([{$group : {_id :
"$by_user", num_tutorial : {$min :
"$likes"}}}])
```

```
db.mycol.aggregate([{$group : {_id :
"$by_user", num_tutorial : {$max :
"$likes"}}}])
```

Aggregate Function

First filter on “AccType:S” and then group it on “CustID” and then compute the sum of “AccBal” and then filter those documents wherein the “TotAccBal” is greater than 1200, use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
{ $match : {TotAccBal : { $gt : 1200 } }});
```

Aggregate Function

Determine Minimum and Maximum account balance for each customer_id.

```
db.Customers.aggregate (
{$group : { _id : "$cust_id",
minAccBal :{$min:"$Balance"},
maxAccBal :{$max:"$Balance"} }});
```

```
{ _id: 2, minAccBal: 50, maxAccBal: 1000 },
{ _id: 1, minAccBal: 200, maxAccBal: 1000 },
{ _id: 3, minAccBal: 500, maxAccBal: 900 }
```

Export data to a CSV file

This command used at the command prompt exports MongoDB JSON documents from “Customers” collection in the “test” database into a CSV file “Output.txt” in the D: drive.

Mongoexport --db test --collection Customers --csv --fieldFile d:\fields.txt --out d:\output.txt

mongoexport mongodb+srv://antararc:<password>@cluster0.mfnfeys.mongodb.net/myDB --collection=Student --out C:\Users\BMSCECSE\Downloads\output.json

```
C:\Users\BMSCECSE>mongoexport mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB  --collection=Student --out C:\Users\BMSCECSE\Downloads\output.json
2023-01-12T14:33:24.184+0530    connected to: mongodb+srv://[**REDACTED**]@cluster0.mfnfeys.mongodb.net/myDB
2023-01-12T14:33:24.301+0530    exported 3 records
```

Import data from a CSV file

Given a CSV file “sample.txt” in the D: drive, import the file into the MongoDB collection, “SampleJSON”.

The collection is in the database “test”.

Mongoimport --db test --collection SampleJSON --type csv --headerline --file d:\sample.txt

mongoimport mongodb+srv://antararc:<password>@cluster0.mfnfeys.mongodb.net/myDB --collection=Student --type json --file C:\Users\BMSCECSE\Downloads\output.json

```
C:\Users\BMSCECSE>mongoimport  mongodb+srv://antararc:Test1234@cluster0.mfnfeys.mongodb.net/myDB --collection=Student  --type json --file C:\Users\BMSCECSE\Downloads\output.json
2023-01-12T14:36:28.113+0530    connected to: mongodb+srv://[**REDACTED**]@cluster0.mfnfeys.mongodb.net/myDB
2023-01-12T14:36:28.290+0530    3 document(s) imported successfully. 0 document(s) failed to import.
```